

# Aula Extra E2 — Redução de Dimensionalidade (PCA e t-SNE)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME cap. 10; ISLP §12.2

## O que você vai aprender

Ao final desta aula você deve ser capaz de:

- listar as razões para reduzir a dimensionalidade;
- definir os **componentes principais** como direções de variância máxima e resolvê-los via autovetores;
- usar a *proporção de variância explicada* para escolher quantos componentes manter;
- interpretar o PCA geometricamente e usá-lo para **compressão**;
- reconhecer as extensões não lineares — **Kernel PCA** e **t-SNE** — e os cuidados com este último.

Esta aula ataca de frente a maldição da dimensionalidade da Aula 05, mas pelo lado da *redundância*: se as covariáveis são correlacionadas, os dados ocupam apenas uma fatia fina do espaço  $\mathbb{R}^d$ , e talvez baste descrevê-la. A pergunta:

*como resumir  $d$  covariáveis em bem menos, sem perder o essencial?*

## 1 Por que reduzir a dimensionalidade?

- **Visualização**: projetar dados em 2D/3D para enxergar estrutura.
- **Compressão**: armazenar/transmitir com menos espaço (imagens).
- **Pré-processamento**: combater a *maldição da dimensionalidade* (Aula 05) e o ruído antes de um método supervisionado.
- **Redundância**: muitas covariáveis correlacionadas vivem perto de uma subvariedade de dimensão menor — queremos recuperá-la.

### Ideia central

O último item é o mais importante conceitualmente, e liga esta aula ao Bloco I. Na Aula 05 vimos que o KNN e a regressão local *se adaptam sozinhos* à dimensão intrínseca  $u$ , mas que métodos que somam todas as coordenadas sofrem com  $d$  grande. O PCA faz esse trabalho *explicitamente*: ele encontra a subvariedade — desde que ela seja aproximadamente um subespaço linear.

## 2 Componentes principais (PCA)

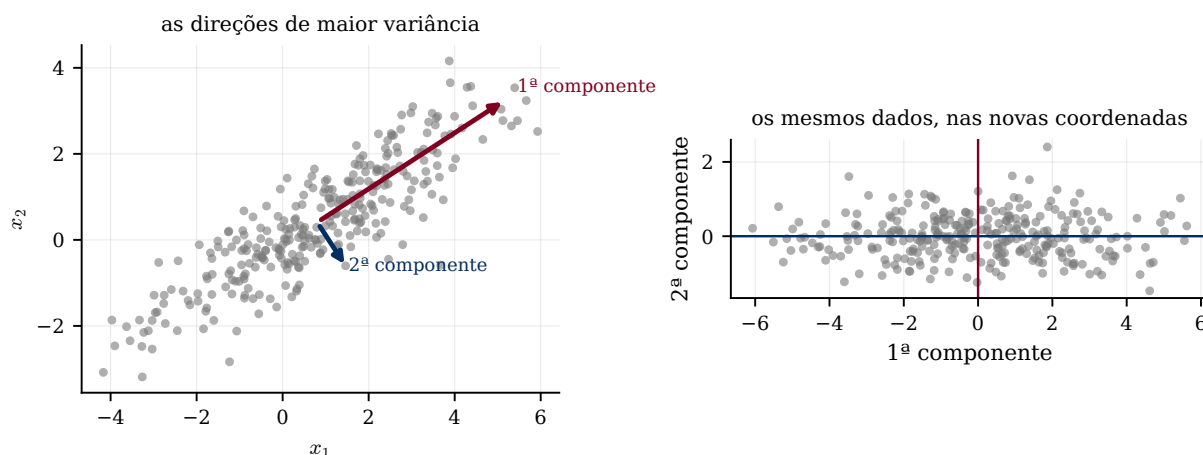
Suponha as covariáveis *centradas* (média zero). O **primeiro componente principal** é a combinação linear

$$Z_1 = \phi_{11}X_1 + \cdots + \phi_{d1}X_d = \phi_1^\top \mathbf{X}$$

de **variância máxima**, sob a restrição  $\|\phi_1\|^2 = \sum_k \phi_{k1}^2 = 1$  (sem ela, bastaria inflar os coeficientes). O  $i$ -ésimo componente  $Z_i$  tem variância máxima entre as combinações lineares unitárias *não correlacionadas* com  $Z_1, \dots, Z_{i-1}$ .

**Ideia central**

Procuramos novos eixos (ortogonais) ao longo dos quais os dados *mais variam*. O 1º eixo capta o máximo de variabilidade; o 2º, o máximo do que resta sendo ortogonal ao 1º; e assim por diante. Concentrar a variância em poucos eixos é o que permite descartar os demais com pouca perda.



**Figura 1:** PCA em duas dimensões. À esquerda, a nuvem original e as duas direções principais, com comprimento proporcional ao desvio-padrão ao longo delas: a primeira acompanha o alongamento da nuvem e responde por 94,4% da variância. À direita, os mesmos pontos escritos nas novas coordenadas — é literalmente uma *rotação*. É aí que a redução fica óbvia: descartar a segunda coordenada custa 5,6% da variância e transforma dois números em um.

## 2.1 Solução: autovetores da covariância

Seja  $\mathbf{C} = \mathbf{X}^\top \mathbf{X}$  (proporcional à matriz de covariância, pois os dados têm média zero). O problema é

$$\max_{\phi} \text{Var}(\phi^\top \mathbf{X}) = \max_{\phi} \phi^\top \mathbf{C} \phi \quad \text{s.a.} \quad \|\phi\|^2 = 1.$$

**Proposição 2.1** (O 1º componente é o autovetor principal). *O maximizador de  $\phi^\top \mathbf{C} \phi$  sob  $\|\phi\| = 1$  é o autovetor de  $\mathbf{C}$  associado ao maior autovalor, e o valor máximo da variância é esse autovalor.*

*Demonstração.* Lagrangiano:  $\mathcal{L}(\phi, \nu) = \phi^\top \mathbf{C} \phi - \nu(\phi^\top \phi - 1)$ . Anulando o gradiente em  $\phi$ :  $2\mathbf{C}\phi - 2\nu\phi = 0$ , isto é,  $\mathbf{C}\phi = \nu\phi$  —  $\phi$  é autovetor de  $\mathbf{C}$  com autovalor  $\nu$ . Para um autovetor unitário, a variância é  $\phi^\top \mathbf{C} \phi = \nu$ . Logo o máximo é atingido no autovetor de *maior* autovalor. Os componentes seguintes saem do mesmo argumento restrito ao complemento ortogonal, gerando os demais autovetores em ordem decrescente de autovalor.  $\square$

**Ideia central**

Vale apreciar quanto essa proposição entrega. O problema original — “ache a melhor direção, depois a melhor entre as ortogonais a ela, e assim por diante” — parece uma sequência de  $d$  otimizações encadeadas. A proposição diz que **uma única** decomposição espectral de  $\mathbf{C}$  resolve todas de uma vez, e ainda ordena o resultado. É por isso que o PCA é barato mesmo com  $d$  grande.

Empilhando os autovetores (ordenados por autovalor decrescente) nas colunas de  $\mathbf{U}$  (a **matriz de cargas**), os *scores* são

$$\mathbf{Z} = \mathbf{X}\mathbf{U} \quad (n \times d),$$

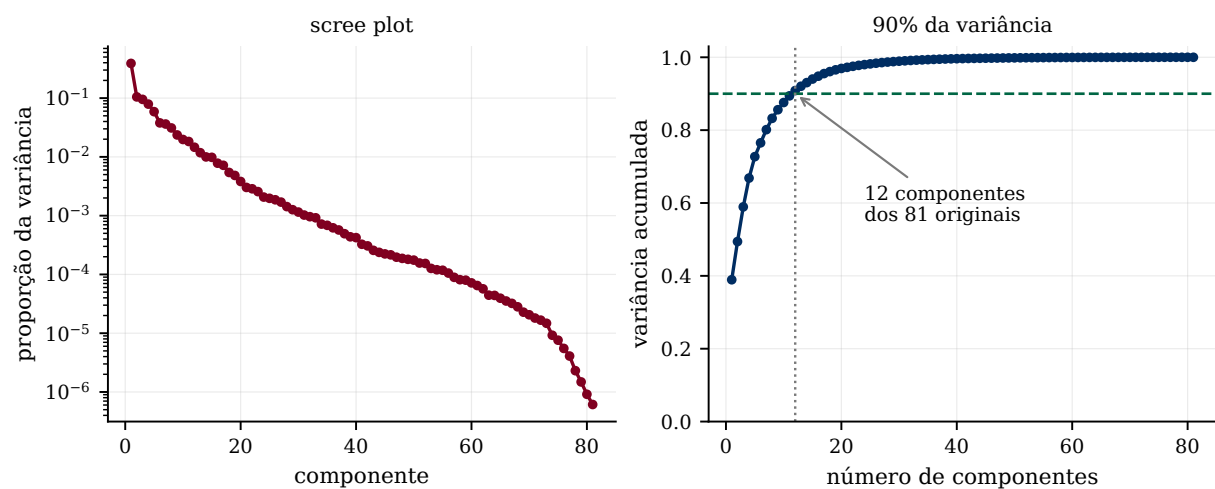
e a  $i$ -ésima nova coordenada da observação  $\mathbf{x}$  é  $z_i = \mathbf{u}_i^\top \mathbf{x}$ . Como os autovetores são ortogonais, os componentes são *não correlacionados* — sem informação redundante.

## 2.2 Quantos componentes? Variância explicada

Cada autovalor  $\lambda_i$  é a variância capturada pelo  $i$ -ésimo componente. A **proporção de variância explicada** pelos  $p$  primeiros é

$$\text{PVE}(p) = \frac{\sum_{i=1}^p \lambda_i}{\sum_{i=1}^d \lambda_i}. \quad (1)$$

Escolhe-se  $p$  olhando o *scree plot* ( $\lambda_i$  vs.  $i$ , procurando o “cotovelo”) ou exigindo PVE acima de um limiar (ex.: 90%).



**Figura 2:** PCA nas 81 covariáveis padronizadas do `superconductivity.csv`, a mesma base das aulas práticas. O primeiro componente sozinho explica 38,9% da variância, e **12 componentes** chegam a 90%. Em outras palavras: as 81 colunas carregam cerca de uma dúzia de dimensões de informação, e as demais 69 são, em boa medida, repetição. É a *redundância* da Aula 05, medida.

## 3 Interpretação geométrica e compressão

### 3.1 PCA como escalonamento multidimensional

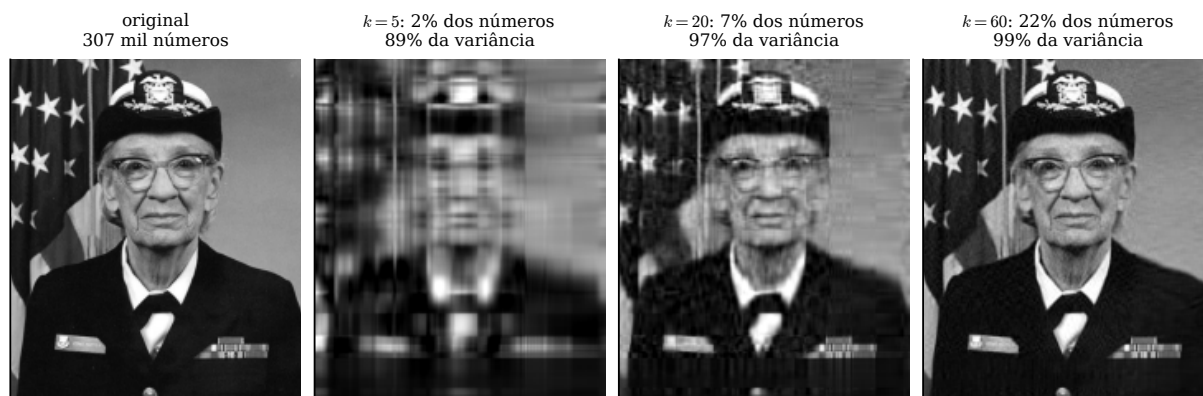
O PCA também resolve: encontrar a projeção linear  $\mathbf{Z}$  em  $m < d$  dimensões que melhor *preserva as distâncias* entre pares,

$$\min_{\mathbf{Z}} \sum_{i,j} (\|\mathbf{x}_i - \mathbf{x}_j\|^2 - \|\mathbf{z}_i - \mathbf{z}_j\|^2).$$

A solução são os  $m$  primeiros componentes principais. Ou seja, PCA é a melhor aproximação linear de baixa dimensão no sentido de distâncias — daí ser tão útil para visualização. Note que são *dois* problemas aparentemente diferentes (maximizar variância, preservar distâncias) com a *mesma* solução; guarde isso, porque o t-SNE vai atacar o segundo problema por outro caminho.

### 3.2 Compressão (SVD truncada)

Da decomposição  $\mathbf{X}^\top \mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{U}^\top$ , reconstrói-se  $\mathbf{X} \approx \mathbf{Z}_p \mathbf{U}_p^\top$ , usando só as  $p$  primeiras colunas. Se os autovalores decaem rápido, a reconstrução é quase perfeita guardando apenas  $(z_{i,1}, \dots, z_{i,p})$  e as  $p$  cargas  $\mathbf{u}_1, \dots, \mathbf{u}_p$  — uma **compressão**.



**Figura 3:** Uma imagem em tons de cinza  $600 \times 512$  (307 mil números) reconstruída a partir de  $k$  componentes. Com  $k = 5$  guardamos 2% dos números e já temos 89% da variância — e a imagem, embora borrada, tem a silhueta certa. Com  $k = 20$  (7% dos números) o rosto é reconhecível; com  $k = 60$  (22%), a diferença para o original quase não se vê. A queda rápida dos autovalores é a razão de tudo isso funcionar.

### Atenção

PCA usa variância e produtos internos  $\Rightarrow$  é **sensível à escala**. *Padronize* as covariáveis antes (Aula 07), senão variáveis de maior magnitude dominam os componentes artificialmente — trocar metros por milímetros numa coluna muda o resultado. E PCA é *não supervisionado*: a direção de maior variância *não* é necessariamente a mais útil para prever um  $Y$ . É perfeitamente possível que o sinal preditivo esteja justamente no componente de menor variância, que você acabou de descartar.

## 4 Além do linear: Kernel PCA e t-SNE

### 4.1 Kernel PCA

PCA só captura estrutura *linear*. Como o PCA pode ser escrito em termos de produtos internos, aplica-se o **truque do kernel** (Aulas 04 e 10): substituem-se os produtos internos por um kernel  $K$ , fazendo PCA num espaço de *features* de alta dimensão. Resultado: componentes **não lineares**, que capturam subvariedades curvas. É literalmente o mesmo truque da Aula 10, aplicado a outro problema — e essa recorrência não é coincidência: qualquer método que só dependa de produtos internos admite a mesma generalização.

### 4.2 t-SNE

O **t-SNE** (*t-distributed Stochastic Neighbor Embedding*) é uma técnica *não linear* feita para **visualização** (2D/3D). A ideia:

- converte distâncias em **probabilidades de vizinhança** no espaço original (kernel gaussiano) e no espaço reduzido (distribuição  $t$  de Student, de caudas pesadas);
- ajusta as posições no espaço reduzido minimizando a divergência de **Kullback–Leibler** entre as duas distribuições;
- assim, *preserva a estrutura local* (vizinhos próximos continuam próximos), revelando agrupamentos.

O principal hiperparâmetro é a **perplexidade** (ordem de grandeza do número de vizinhos considerados).

**Ideia central: por que a distribuição  $t$** 

O detalhe do nome tem uma razão precisa. Ao espremer dados de  $\mathbb{R}^d$  em  $\mathbb{R}^2$ , não há espaço para manter todo mundo afastado — pontos moderadamente distantes seriam esmagados uns sobre os outros. A distribuição  $t$ , com caudas pesadas, atribui probabilidade razoável a distâncias grandes no espaço reduzido, o que permite ao algoritmo *espalhar* os grupos em vez de amontoá-los. É uma solução para um problema de espaço, não de estatística.

**Atenção: Cuidados com o t-SNE**

- é **só para visualização** — não use os eixos como *features* para um modelo supervisionado (e note que, ao contrário do PCA, ele nem tem um **transform** para aplicar a dados novos);
- **distâncias e tamanhos** entre grupos no gráfico *não* têm significado — só a vizinhança local é confiável. Dois grupos afastados na figura não são necessariamente mais diferentes que dois próximos;
- é **estocástico** e sensível à perplexidade: rode mais de uma vez e compare. Se os agrupamentos mudam de uma rodada para outra, eles não estão nos dados. Para grandes bases, UMAP é uma alternativa mais rápida.

Em resumo: PCA para *compressão e pré-processamento* (linear, interpretável, com **transform** aplicável a dados novos); t-SNE para *olhar* os dados, e só.

## 5 Prática em Python

```

1 from sklearn.decomposition import PCA
2 from sklearn.manifold import TSNE
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.pipeline import make_pipeline
5
6 # PCA como etapa de um pipeline (Aula 07): ajustado SO no treino
7 pipe = make_pipeline(StandardScaler(), PCA(n_components=0.90), Ridge())
8
9 # quanta variancia cada componente explica
10 pca = PCA().fit(StandardScaler().fit_transform(X))
11 print(pca.explained_variance_ratio_.cumsum())
12
13 # t-SNE: apenas para visualizar
14 Z = TSNE(n_components=2, perplexity=30, random_state=0).fit_transform(X)
```

O truque do `n_components=0.90` é útil: em vez de um número de componentes, passa-se a *proporção de variância* desejada, e o `scikit-learn` escolhe  $p$  sozinho.

**Atenção**

O PCA dentro de um *pipeline* não é preciosismo: os componentes são direções aprendidas de *todas* as colunas ao mesmo tempo, e calculá-los fora da dobra é vazamento da categoria grave da Aula 07 — da mesma família da seleção de variáveis que fabricou  $R^2 = 0,40$  a partir de ruído.

Os notebooks Aula prática E2 e Exemplo - PCA ilustram os componentes principais e a variância explicada em dados reais.

## Resumo

- Reduzir dimensão serve para visualizar, comprimir, pré-processar e explorar *redundância* (Aula 05).
- **PCA**: direções ortogonais de variância máxima; são os autovetores de  $\mathbf{X}^\top \mathbf{X}$  em ordem de autovalor (Prop. 2.1), obtidos de uma única decomposição (Fig. 1).
- *Proporção de variância explicada* (1) guia a escolha de  $p$ : no **superconductivity**, 12 dos 81 componentes dão 90% (Fig. 2).
- PCA = melhor projeção linear no sentido de distâncias; a truncagem da SVD é compressão (Fig. 3).
- *Padronize* antes; e lembre que PCA é não supervisionado — a direção de maior variância pode não ser a mais preditiva.
- **Kernel PCA** e **t-SNE** estendem para o não linear; o t-SNE é *só* para visualização, e distâncias entre grupos nele não significam nada.

**Leitura recomendada.** [AME] Capítulo 10: §10.1 (PCA e a interpretação como escalonamento multidimensional), §10.1.2 (compressão de imagens, que é a nossa Figura 3), §10.2 (Kernel PCA) e §10.4 (*autoencoders*, a generalização não linear via redes neurais). [ISLP] §12.2 — §12.2.1–12.2.2 (o que são os componentes e a interpretação alternativa), §12.2.3 (proporção de variância explicada, a nossa Figura 2) e §12.2.4, que discute honestamente quantos componentes usar e por que não há resposta objetiva.